Kubernetes DaemonSets Overview

Alright freshers, welcome to **Kubernetes survival school** 😄
I'll explain this like a Kubernetes admin who has *broken production at least once* so you don't have to.

We'll go **one by one** as you asked:

1️⃣ DaemonSet

Step 1 – Overview (High-level understanding)

A **DaemonSet** makes sure that **a pod runs on every node** (or selected nodes) in the cluster.

👉 Think of it as:

> "Install this software on every machine, no excuses."

Used for:

- Log collectors
- Monitoring agents
- Security agents
- Networking components

If a new node joins → pod is automatically added
If a node is removed → pod is removed
Like a clingy friend, but useful.

Step 2 – Core Concepts

- One pod per node
- No need to specify replicas
- Automatically handles node changes
- Uses `nodeSelector`, `taints`, `tolerations`

Step 3 – Hands-on Example (DaemonSet YAML)

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: log-agent
spec:
  selector:
    matchLabels:
      app: log-agent
  template:
    metadata:
      labels:
        app: log-agent
    spec:
      containers:
      - name: log-agent
        image: busybox
        command: ["sh", "-c", "while true; do echo collecting logs; sleep 10; done"]
        # This pod will run on EVERY node
```

Apply:

```bash
kubectl apply -f daemonset.yaml
```

Check:

```bash
kubectl get pods -o wide
```

You'll see **one pod per node**. Magic 🪄

Step 4 – Common Mistakes

❌ Using DaemonSet for applications (use Deployment instead)
❌ Forgetting tolerations for master nodes
❌ Expecting replicas to work (they don't)
❌ Using it for workloads that don't need to run everywhere

Step 5 – Advanced Usage / Optimization

- Run only on specific nodes:

```yaml
nodeSelector:
  node-type: worker
```

- Run on master nodes:

```yaml
tolerations:
- key: node-role.kubernetes.io/control-plane
  operator: Exists
  effect: NoSchedule
```

- Use for:

- Fluentd
- Prometheus Node Exporter
- CNI plugins
- Security scanning agents

2️⃣ Job

Step 1 – Overview (High-level understanding)

A **Job** runs a pod **until completion**.
It's not a service, it's a task.

👉 Think of it as:

> "Run this, finish it, and then disappear like Batman."

Used for:

- Database migration
- Backup
- One-time scripts
- Batch processing

Step 2 – Core Concepts

- Runs to completion (success/failure)
- Retries on failure
- Can run multiple pods in parallel
- Controlled by `completions` and `backoffLimit`

Step 3 – Hands-on Example (Job YAML)

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: hello-job
spec:
  completions: 1   # run once
  backoffLimit: 3  # retry 3 times on failure
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: hello
        image: busybox
        command: ["sh", "-c", "echo Hello Kubernetes Job; sleep 5"]
        # This job runs once and exits
```

Apply:

```bash
kubectl apply -f job.yaml
```

Check:

```bash
kubectl get jobs
kubectl get pods
```

Once completed → pod stops.

Step 4 – Common Mistakes

❌ Using Job for long-running apps
❌ Using `restartPolicy: Always` (illegal here)
❌ Forgetting backoff limit → infinite pain
❌ Not cleaning completed jobs

Step 5 – Advanced Usage / Optimization

Parallel jobs:

```yaml
parallelism: 5
completions: 5
```

Auto-cleanup:

```yaml
ttlSecondsAfterFinished: 60
```

Perfect for:

- ETL jobs
- Reports
- Batch processing
- Migrations

3️⃣ CronJob

Step 1 – Overview (High-level understanding)

A **CronJob** is just a **Job with a schedule**.

👉 Think of it as:

> "Run this job every day at 2 AM while everyone sleeps and no one can blame you."

Used for:

- Backups
- Cleanup tasks
- Reports
- Sync jobs

Step 2 – Core Concepts

- Uses Linux cron syntax
- Creates Jobs on schedule
- Jobs create Pods
- History can be limited

Step 3 – Hands-on Example (CronJob YAML)

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
spec:
  schedule: "*/1 * * * *"  # every 1 minute (for demo)
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
```

```yaml
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
          - name: backup
            image: busybox
            command: ["sh", "-c", "echo Backup started; sleep 10"]
            # Runs every minute
```

Apply:

```bash
kubectl apply -f cronjob.yaml
```

Check:

```bash
kubectl get cronjobs
kubectl get jobs
```

Step 4 – Common Mistakes

❌ Wrong cron syntax (classic 3 AM disaster)
❌ Jobs overlapping (use concurrencyPolicy)
❌ Not limiting job history
❌ Running heavy jobs too frequently

Step 5 – Advanced Usage / Optimization

Prevent overlap:

```yaml
concurrencyPolicy: Forbid
```

Replace running job:

```yaml
concurrencyPolicy: Replace
```

Suspend temporarily:

```yaml
suspend: true
```

Timezone support (newer versions):

```yaml
timeZone: "Asia/Kolkata"
```

🧠 Final Memory Trick

Resource	Purpose
DaemonSet	Run on every node
Job	Run once and exit
CronJob	Run on schedule

If you want, I can:

- Make this into **notes PDF**
- Add **interview questions**
- Convert to **lab exercises**
- Or explain like I'm talking to a 10-year-old 😄

Just say the word, admin-in-training 👩‍💻🔥